



Source Program (PyTorch)

```
import torch

import torch.nn as nn

class Net(nn.Module):

    def __init__(self):
        super().__init__()
        self.fc = nn.Linear(128, 10)

    def forward(self, x):
        return self.fc(x)

# training loop

model = Net().to(device)

for x, y in loader:
    loss = criterion(model(x), y)
    loss.backward()
    optimizer.step()
```



- ✓ Migrate to target framework(s)
- ✓ Keep training behavior
- ✓ Fix incompatibilities and errors

Iterative Repair for Cross-Framework Migration



3 / 10



initial candidate

diagnosis / feedback

targeted repair



Initial Target Program

```
import mindspore as ms
import mindspore.nn as nn

class Net(nn.Cell):
    def __init__(self):
        super().__init__()
        self.fc = nn.Dense(128, 10)
    def construct(self, x):
        return self.fc(x)
```



Layered Diagnosis(example)



```
graph TD; A[✓ Execution] --> B[✓ Forward]; B --> C[! Gradients & Updates];
```

Execution

Forward

Gradients & Updates
First failing stage + evidence

RuntimeError: gradient does not exist for parameter 'fc.weight'

Later Training Behavior



Revised Target Program

```
import mindspore as ms
import mindspore.nn as nn

class Net(nn.Cell):

    .....

    def train_step(self, x, y):
        loss = self.loss_fn(x, y)
        return loss
```

 Targeted patching with framework-aware expertise

Updated candidate (next round)



Verified Target Programs

- ✓ Pass verification
- ✓ Preserve training behavior
- ✓ Ready for downstream use